

Mentor-UADB

Cahier des charges

Application de gestion du mentorat étudiant

Projet	Mentor-UADB
Porteur	Université Alioune Diop de Bambey (UADB)
Contexte	Projet de fin d'études — cours de Java avancé
Version	1.0
Date	04/10/2026
Statut	Version 1.0 déposée sur GitHub
Technologies	Java 21 · JavaFX 21 · JDBC · MySQL 8 · Maven

Document de spécifications — Cahier des charges

Table des matières

1. Contexte et justification
2. Objectifs
3. Périmètre du projet
4. Parties prenantes
5. Exigences fonctionnelles
6. Règles métier

7. Exigences non fonctionnelles
8. Modèle de données
9. Architecture et choix techniques
- 10.Écrans et parcours
- 11.Cas d'usage principaux
- 12.Sécurité
- 13.Limites connues et hors périmètre
- 14.Planning réalisé
- 15.Livrables
- 16.Critères d'acceptation
- 17.Organisation de l'équipe

1. Contexte et justification

L'UADB ouvre chaque année des campus d'été et des initiatives de mentorat entre étudiants. Aujourd'hui, la mise en relation repose sur des canaux informels (groupes de discussion, affiches, bouche-à-oreille), ce qui pose trois problèmes :

- **transparence** : un étudiant ne sait pas qui peut réellement l'aider dans sa matière, ni sur quel créneau ;
- **traçabilité** : aucune histoire des séances, donc aucune progression mesurable ni évaluation ;
- **contrôle** : l'administration ne dispose d'aucun moyen de vérifier les profils de mentors ni de piloter les données de référence (UFR, filières, niveaux, matières).

L'application centralise ces trois aspects : un étudiant trouve un mentor validé et lui envoie une demande, le mentor organise des séances, les deux sont évalués, et l'administration supervise tout le processus.

2. Objectifs

2.1 Objectif général

Concevoir et réaliser une application desktop permettant de gérer l'intégralité du processus de mentorat étudiant au sein de l'UADB, de la recherche du mentor jusqu'à l'évaluation des séances.

2.2 Objectifs spécifiques

N°	Objectif	Critère de réussite
----	----------	---------------------

OS-1 Permettre à un étudiant de trouver un mentor validé Recherche fonctionnelle avec résultats

N°	Objectif	Critère de réussite
	correspondant à sa filière, son niveau et sa matière	notés et filtrés
OS-2	Permettre à un mentor de valider son profil et d'accepter ou refuser des demandes	Statuts EN_ATTENTE / VALIDE / REFUSE gérés
OS-3	Permettre l'organisation et le suivi des séances individuelles et de groupe	Séances planifiées, réalisées, annulées
OS-4	Permettre l'évaluation des séances avec note et commentaire	Note de 1 à 5, moyenne affichée par mentor
OS-5	Donner à l'administration un contrôle complet sur les utilisateurs et les données de référence	CRUD complet sur UFR, filières, niveaux, matières, utilisateurs
OS-6	Sécuriser l'accès aux données par authentification et contrôle d'accès par rôle	3 rôles, mots de passe jamais stockés en clair
OS-7	Produire une application maintenable et collaborative	Code versionné sur GitHub, architecture en couches

3. Périmètre du projet

3.1 Dans le périmètre

- Authentification par email institutionnel et mot de passe
- Gestion des rôles **étudiant**, **mentor**, **administrateur**
- Inscription autonome des étudiants
- Référentiel académique : UFR, filières, niveaux, matières
- Recherche de mentors par filière / niveau / matière, avec note moyenne
- Demande de mentorat, acceptation ou refus
- Demande de séance, planification, confirmation, annulation
- Séances individuelles et séances de groupe
- Partage de fichiers (supports) par les mentors
- Évaluation des séances (note + commentaire)
- Notifications internes aux événements du workflow
- Administration : utilisateurs et données de référence
- Interface graphique JavaFX avec charte visuelle bleue/blanche

3.2 Hors périmètre (explicitement exclu)

- Application web ou mobile : le livrable est une application **desktop**
- Envoi d'emails reels (notifications par email, lien de réinitialisation)
- Paiement, contractualisation ou certification des mentors
- Visioconférence intégrée : la séance en visio repose sur un lien fourni par le mentor
- Import/export de données, sauvegarde automatique, réplication
- Mode hors ligne / synchronisation
- Authentification à deux facteurs
- Tableau de bord de statistiques avancé et export PDF

4. Parties prenantes

Partie prenante	Besoins	Accès concedes
Étudiant UADB	Trouver un mentor pour sa matière, organiser des séances, être évalué, suivre sa progression	Écrans : profil, recherche, tableau de bord
Mentor	Se rendre visible, gérer ses demandes, planifier ses séances, partager des supports, évaluer	Écrans : tableau de bord mentor, profil
Administration UADB	Valider les profils mentor, gérer les comptes et le référentiel, superviser l'ensemble	Écrans : validation, liste des mentors, utilisateurs, données
Maintenance / équipe de développement	Code lisible, versionné, testable	Architecture en couches, dépôt GitHub

5. Exigences fonctionnelles

5.1 Authentification et comptes

Réf.	Exigence
EF-AUTH-01	L'application doit proposer un écran d'accueil puis un écran de connexion
EF-AUTH-02	La connexion doit s'effectuer par email institutionnel et mot de passe

Réf.	Exigence
EF-AUTH-03	Le système doit rediriger l'utilisateur vers son tableau de bord selon son rôle
EF-AUTH-04	Un utilisateur inactif ne doit pas pouvoir se connecter
EF-AUTH-05	L'étudiant doit pouvoir s'inscrire de façon autonome (nom, prénom, email, téléphone, numéro de carte, filière, niveau, mot de passe)
EF-AUTH-06	L'email doit être unique et de domaine institutionnel @uadb.edu.sn
EF-AUTH-07	L'étudiant inscrit doit recevoir un message de confirmation puis être redirigé vers la connexion
EF-AUTH-08	Un utilisateur doit pouvoir demander la réinitialisation de son mot de passe via son email
EF-AUTH-09	Toute déconnexion doit ramener à l'écran de connexion
EF-AUTH-10	L'écran de connexion doit offrir un retour vers l'écran d'accueil

5.2 Espaces étudiant

Réf.	Exigence
EF-ETU-01	L'étudiant doit pouvoir consulter et modifier son profil (téléphone, numéro de carte, photo)
EF-ETU-02	L'étudiant doit pouvoir rechercher des mentors par filière, niveau et matière
EF-ETU-03	Les résultats de recherche doivent afficher le mentor, sa filière, son niveau, ses matières et sa note moyenne
EF-ETU-04	L'étudiant doit pouvoir consulter le profil détaillé d'un mentor
EF-ETU-05	L'étudiant doit pouvoir envoyer une demande de mentorat sur une matière qu'un mentor maîtrise effectivement
EF-ETU-06	L'étudiant doit pouvoir demander une séance auprès d'un mentor avec lequel une demande a été acceptée
EF-ETU-07	L'étudiant doit pouvoir confirmer sa présence à une séance

Réf.	Exigence
------	----------

EF-ETU-08 L'étudiant doit pouvoir annuler une demande de séance qu'il a émise

EF-ETU-09 L'étudiant doit pouvoir masquer une demande ou une séance de son propre tableau de bord

EF-ETU-10 L'étudiant doit pouvoir consulter ses notifications

5.3 Espace mentor

Réf.	Exigence
EF-MEN-01	Un étudiant doit pouvoir déposer une candidature mentor (biographie, expérience, mode de préférence, nombre maximal de mentorés)
EF-MEN-02	Le système doit attribuer le statut <code>EN_ATTENTE</code> à la candidature
EF-MEN-03	L'étudiant doit voir l'état de sa candidature sur son profil
EF-MEN-04	Le mentor doit voir les demandes de mentorat reçues, avec demandeur et matière
EF-MEN-05	Le mentor doit pouvoir accepter ou refuser une demande, une seule fois
EF-MEN-06	Le mentor doit voir ses demandes de séance et y répondre (accepter avec date et heure, ou refuser)
EF-MEN-07	Le mentor doit pouvoir marquer une séance comme réalisée
EF-MEN-08	Le mentor doit pouvoir annuler une séance
EF-MEN-09	Le mentor doit pouvoir partager un fichier lié à une matière ou à une séance
EF-MEN-10	Le mentor doit pouvoir évaluer une séance réalisée (note de 1 à 5 et commentaire), une seule fois par étudiant

Réf.	Exigence
EF-MEN-11	Le mentor doit pouvoir modifier son profil

5.4 Espace administration

Réf.	Exigence
EF-ADM-01	L'administrateur doit disposer d'un tableau de bord donnant accès à toutes les fonctions d'administration
EF-ADM-02	L'administrateur doit consulter la liste des candidatures mentor et ouvrir un dossier complet (identité, email, téléphone, numéro de carte, filière, niveau, expérience, mode, capacité, biographie)
EF-ADM-03	L'administrateur doit valider ou refuser une candidature ; les actions doivent être inactives tant qu'aucune candidature n'est sélectionnée
EF-ADM-04	L'administrateur doit consulter la liste de tous les mentors, quel que soit leur statut
EF-ADM-05	La liste des mentors doit permettre un filtre par statut (validés, en attente, refusés) et afficher le statut coloré
EF-ADM-06	L'administrateur doit consulter la fiche détaillée d'un mentor depuis la liste
EF-ADM-07	L'administrateur doit gérer les utilisateurs : lister, activer, désactiver, supprimer
EF-ADM-08	Un compte administrateur ne doit pas pouvoir être désactivé ni supprimé
EF-ADM-09	Un utilisateur possédant des données liées ne doit pas pouvoir être supprimé
EF-ADM-10	L'administrateur doit gérer les UFR (création, suppression)
EF-ADM-11	L'administrateur doit gérer les filières rattachées à une UFR

Réf.	Exigence
EF-ADM-12	L'administrateur doit gérer les niveaux
EF-ADM-13	L'administrateur doit rattacher des niveaux à des filières et des matières à des filières
EF-ADM-14	L'administrateur doit supprimer un mentor, une matière ou une évaluation uniquement si aucune donnée n'y est rattachée

5.5 Écrans

L'application compte **15 écrans** :

[bienvenue](#) · [login](#) · [inscription](#) · [mot-de-passe-oublie](#) · [dashboard-etudiant](#) · [dashboard-mentor](#) · [dashboard-admin](#) · [profil](#) · [recherche-mentor](#) · [devenir-mentor](#) · [seance-groupe](#) · [validation-mentors](#) · [liste-mentors](#) · [gestion-utilisateurs](#) · [gestion-donnees](#)

6. Règles métier

Réf.	Règle
RM-01	Un mentor doit être au statut VALIDE pour apparaître dans les résultats de recherche
RM-02	Un mentor ne peut pas être son propre mentorat
RM-03	Une demande de mentorat ne peut porter que sur une matière effectivement maîtrisée par le mentor
RM-04	Une demande de mentorat ne peut être traitée (acceptée ou refusée) qu'une seule fois
RM-05	Une demande de séance ne peut être émise que dans le cadre d'une relation de mentorat acceptée
RM-06	Une demande de séance ne peut être traitée qu'une seule fois
RM-07	Seule une séance réalisée peut être évaluée

Réf.	Règle
RM-0 8	Une note doit être comprise entre 1 et 5
RM-0 9	Un étudiant ne peut évaluer qu'une seule fois une même séance
RM-1 0	Le mentor ne peut évaluer que les séances auxquelles il participe
RM-1 1	Un compte <code>ADMIN</code> ne peut être ni désactivé ni supprimé
RM-1 2	Une suppression est refusée si des données dépendantes existent (mentorat, étudiants, évaluation, filières)
RM-1 3	Un format d'heure non conforme à <code>HH:mm</code> est refusé
RM-1 4	Le masquage d'une demande ou d'une séance par un utilisateur est propre à cet utilisateur : les données restent en base et restent visibles par l'administration
RM-1 5	Les notifications sont générées à chaque événement du workflow (demande créée, acceptée, refusée, séance demandée, planifiée, refusée)

7. Exigences non fonctionnelles

7.1 Technologies imposées

Exigence	Valeur
Langage	Java 21
Interface graphique	JavaFX 21.0.2 avec FXML et CSS
Base de données	MySQL 8 (connecteur <code>mysql-connector-j</code> 8.3.0)
Accès aux données	JDBC avec <code>PreparedStatement</code> exclusivement
Build	Maven 3.9+
Versionnement	Git, dépôt GitHub, branche <code>main</code>

7.2 Qualité

Réf.	Exigence
ENF-01	Aucune requête SQL ne doit être construite par concaténation de chaînes : tout paramètre doit être lié (?)
ENF-02	Les mots de passe ne doivent jamais être stockés en clair
ENF-03	Aucun identifiant de connexion ne doit être écrit en dur dans le code
ENF-04	Toute erreur attendue doit être traitée et signalée à l'utilisateur par un message lisible
ENF-05	L'interface doit être écrite dans une seule charte visuelle (couleurs, espacements, largeurs de boutons, hiérarchie des titres)
ENF-06	Aucun texte de bouton ne doit être tronqué
ENF-07	Les fenêtres doivent avoir une taille fixe de 950 × 700 px

7.3 Maintenabilité

Réf.	Exigence
ENF-08	L'architecture doit être en couches : JavaFX/FXML → Controllers → Services → DAO → JDBC → MySQL
ENF-09	Chaque entité doit disposer d'une classe modèle et d'une classe DAO
ENF-10	Les vues d'affichage enrichies par jointion doivent être isolées dans des classes DTO dédiées
ENF-11	Le code doit être versionné sur GitHub avec des commits explicites

7.4 Performance

Réf.	Exigence
ENF-12	Un écran doit s'afficher sans latence perceptible ; les listes doivent charger en moins de 2 secondes sur le jeu de données de démonstration
ENF-13	Les requêtes de liste doivent être limitées aux colonnes nécessaires et indexées sur les clés

Réf.	Exigence
3	étrangères

8. Modèle de données

La base `mentoradb` comporte **17 tables**. Les clés étrangères sont garanties par des contraintes d'intégrité, et les suppressions sont protégées par l'application (règle RM-12) pour restituer un message explicite plutôt qu'une erreur SQL.

8.1 Tables

Table	Rôle	Colonnes principales
<code>utilisateur</code>	Compte et rôle	nom, prenom, email, mot_de_passe, role, statut, telephone, photo
<code>ufr</code>	Unités de recherche et formation	nom
<code>filiere</code>	Filières d'enseignement	nom, id_ufr
<code>niveau</code>	Niveaux d'études	libelle
<code>filiere_niveau</code>	Association filière / niveau	id_filiere, id_niveau
<code>etudiant</code>	Profil étudiant	id_utilisateur, id_filiere, id_niveau, numero_carte, choisi
<code>matiere</code>	Matières enseignées	nom
<code>filiere_matiere</code>	Association filière / matière	id_filiere, id_matiere
<code>mentor</code>	Profil mentor	id_etudiant, statut_validation, biographie, experience, mode_preference, nombre_max_mentores
<code>expertise</code>	Matières maîtrisées par un mentor	id_mentor, id_matiere
<code>demande_mentorat</code>	Demande de mentorat	id_etudiant, id_mentor, id_matiere, date_demande, statut, masque_etudiant, masque_mentor
<code>demande_seance</code>	Demande de séance	id_demande, date_souhaitee, message, statut, date_creation,

Table	Rôle	Colonnes principales
		id_seance, masques
seance	Séance planifiée	id_demande, type_seance, id_mentor, id_matiere, date_seance, heure_debut, heure_fin, statut, modalite, lieu, masques
seance_participant	Étudiants participants	id_seance, id_etudiant, confirme, vu, masque
evaluation	Évaluation d'une séance	id_seance, id_etudiant, note, commentaire, date_evaluation, masque_mentor
notification	Notifications internes	id_utilisateur, message, lue, date_creation
fichier	Fichiers partagés	id_mentor, id_matiere, id_seance, id_etudiant, nom_fichier, chemin, date_upload

8.2 Nuancier contrôlé

Nuancier	Valeurs
utilisateur.role	ETUDIANT, MENTOR, ADMIN
utilisateur.statut	ACTIF, INACTIF
mentor.statut_validation	EN_ATTENTE, VALIDE, REFUSE
demande_mentorat.statut	EN_ATTENTE, ACCEPTEE, REFUSEE
demande_seance.statut	EN_ATTENTE, ACCEPTEE, REFUSEE
seance.type_seance	INDIVIDUELLE, GROUPE
seance.statut	PLANIFIEE, REALISEE, ANNULEE
seance.modalite	PRESENTIEL, EN_LIGNE
evaluation.note	entier de 1 à 5

9. Architecture et choix techniques

9.1 Architecture en couches

```

`  JavaFX (FXML + CSS) ↓ Controllers (15 contrôleurs, un par écran) ↓ Services (AuthService,
InscriptionService, MentoratService) ↓ DAO (17 classes, une par entité) ↓ JDBC (PreparedStatement) ↓

```

9.2 Justification des choix

Choix	Justification
JavaFX + FXML	Séparation nette entre la vue (FXML) et la logique (contrôleurs). L'interface est modifiable sans toucher au code Java.
Architecture en couches	Chaque couche a une responsabilité unique. Le DAO n'accède qu'aux données, le Service porte les règles métier, le Contrôleur orchestre l'écran. Testable et maintenable.
JDBC direct	Choix imposé par l'enseignement du cours Java / JDBC. Permet de maîtriser les transactions et les <code>PreparedStatement</code> .
Classes séparées	DTO Les vues d'affichage (mentor avec identité, matière, note moyenne) sont distinctes des entités pour ne pas coupler le modèle à l'affichage.
PBKDF2-HMAC-SHA256	Algorithme de dérivation de clé recommandé pour le stockage de mots de passe, avec sel aléatoire par compte.
Configuration externalisée	Les identifiants de base de données sont lus dans l'ordre : variables d'environnement, puis <code>db.properties</code> (ignoré par Git), puis valeurs par défaut. Aucun secret dans le code.
Scripts versionnés	SQL <code>schema.sql</code> , <code>data.sql</code> et 3 scripts de migration permettent de reconstruire la base et de faire évoluer le schéma de façon traçable.

9.3 Organisation physique du code

Package	Contenu
<code>Main.java</code>	Point d'entrée JavaFX, chargement de l'écran d'accueil
<code>config/</code>	<code>DatabaseConnection</code> — connexion JDBC et lecture de la configuration
<code>model/</code>	12 entités Java
<code>dao/</code>	17 classes d'accès aux données
<code>dto/</code>	11 vues d'affichage enrichies
<code>service/</code>	3 services métier

Package	Contenu
---------	---------

`controller/` 15 contrôleurs FXML

`util/` `PasswordUtil` (hachage), `SceneManager` (navigation)

Volume total : environ 7 800 lignes de Java et 920 lignes de FXML.

10. Écrans et parcours

10.1 Écran d'accueil

- Fond dégradé bleu, texte et cartes en blanc et bleu clair
- Logo UADB, titre « Mentor-UADB », sous-titre « La plateforme de mentorat de l'UADB »
- Trois cartes interactives : **Trouver un mentor**, **Devenir mentor**, **Suivre le mentorat**, chacune redirigeant vers l'écran correspondant
- Deux boutons : **Se connecter**, **Créer un compte**

10.2 Connexion

- Email institutionnel, mot de passe, affichage masqué/visible du mot de passe
- Boutons : **Se connecter**, **Créer un compte**, **Mot de passe oublié ?**
- Lien **Retour à l'accueil** positionné dans le coin haut gauche, sous forme de lien bleu sur fond transparent

10.3 Inscription et mot de passe oublié

- Formulaire en deux étapes : identité, puis filière et niveau
- Validation du domaine `@uadb.edu.sn` et de l'unicité de l'email
- Réinitialisation par email, avec message de confirmation

10.4 Tableau de bord étudiant

Demandes de mentorat, demandes de séance, notifications, accès au profil, à la recherche de mentor et à la candidature mentor.

10.5 Tableau de bord mentor

Demandes reçues, demandes de séance à traiter, séances à confirmer, évaluations, partage de fichiers, gestion des séances de groupe.

10.6 Tableau de bord administration

Quatre entrées : validation des candidats, liste des mentors, gestion des utilisateurs, gestion des données de référence.

10.7 Dossier de validation (administration)

Tableau des candidatures **et** dossier détaillé du candidat sélectionné : identité, contact, formation, expérience, mode, capacité, biographie. Les boutons **Valider** et **Refuser** sont inactifs tant qu'aucune candidature n'est sélectionnée.

10.8 Liste des mentors (administration)

Tous les mentors avec filtre par statut, statuts colorés et ouverture de la fiche détaillée.

10.9 Gestion des utilisateurs et des données

- Utilisateurs : liste, activation, désactivation, suppression protégée
- Données : arbre UFR → Filières → Matières / Niveaux, avec création, suppression, rattachement et détachement

11. Cas d'usage principaux

UC-01 — Trouver et contacter un mentor

Acteur : étudiant · **Précondition** : être connecté

1. L'étudiant ouvre « Recherche de mentor ».
2. Il choisit une filière, un niveau et une matière.
3. Le système affiche les mentors validés correspondant aux critères, avec leur note moyenne.
4. L'étudiant ouvre un profil, lit la présentation et les matières.
5. Il envoie une demande de mentorat.
6. Le système vérifie que le mentor maîtrise la matière (RM-03) et notifie le mentor.

Exception : si la matière n'est pas maîtrisée, la demande est refusée avec un message explicite.

UC-02 — Candidater comme mentor

Acteur : étudiant · **Précondition** : être connecté

1. L'étudiant remplit sa biographie, son expérience, son mode de préférence et le nombre maximal de mentorés.
2. Il dépose sa candidature.

3. Le système crée le profil mentor au statut **EN_ATTENTE** et son profil affiche la candidature en attente.

Exception : le profil ne peut être complété qu'une fois la candidature déposée.

UC-03 — Valider une candidature mentor

Acteur : administrateur · **Précondition** : être connecté en administrateur

1. L'administrateur ouvre « Valider les candidats mentor ».
2. Il voit la liste des candidatures **EN_ATTENTE**.
3. Il sélectionne une candidature : le dossier complet s'affiche à droite.
4. Il clique sur **Valider** ou **Refuser**.
5. Le statut est mis à jour et l'étudiant reçoit une notification.

Exception : sans sélection, les deux boutons sont inactifs.

UC-04 — Organiser une séance

Acteur : étudiant puis mentor

1. L'étudiant, dont la demande de mentorat a été acceptée, demande une séance avec une date souhaitée et un message.
2. Le mentor reçoit une notification et voit la demande dans son tableau de bord.
3. Il accepte en fixant la date, l'heure de début, l'heure de fin, la modalité et le lieu.
4. La séance passe au statut **PLANIFIEE** et l'étudiant est notifié.
5. Après la séance, le mentor la marque **REALISEE**.
6. L'étudiant évalue la séance : note de 1 à 5 et commentaire.

Exceptions : demande sur une relation non acceptée (RM-05), demande déjà traitée (RM-06), séance non réalisée (RM-07), note hors bornes (RM-08), double évaluation (RM-09).

UC-05 — Administrer les données de référence

Acteur : administrateur

1. L'administrateur ouvre « Gestion des données ».
2. Il sélectionne une UFR ou une filière dans l'arbre.
3. Il crée, supprime ou rattache niveaux et matières.
4. Le système refuse la suppression si des données en dépendent (RM-12) et l'explique.

12. Sécurité

Mesure	Mise en œuvre
Mots de passe	PBKDF2-HMAC-SHA256, sel aléatoire de 16 octets par compte, format stocké <code>pbkdf2\$<itérations>\$<sel>\$<hash></code>
Injection SQL	100 % des requêtes utilisent <code>PreparedStatement</code> ; aucun assemblage de chaînes SQL
Comptes démonstration	de Les comptes de <code>data.sql</code> sont en clair dans le script et convertis en hachage à leur première connexion
Identifiants connexion	de Lus depuis l'environnement ou <code>db.properties</code> ; aucun secret dans le code ; <code>db.properties</code> ignoré par Git
Contrôle d'accès	Trois rôles, redirections distinctes, actions réservées au rôle concerné
Protection comptes	des Un administrateur ne peut pas être désactivé ni supprimé
Protection données sensibles	des Les secrets ne sont jamais stockés en clair : ni mots de passe, ni identifiants de connexion. Seules les données nécessaires au fonctionnement sont conservées.

13. Limites connues et hors périmètre

Ces limites sont **assumées et documentées**, et non des défauts bloquants pour le périmètre du projet.

N°	Limite	Explication
L-0 1	La réinitialisation de mot de passe ne vérifie pas l'identité	Il suffit de connaître l'email institutionnel. Une version réelle enverrait un lien ou un code par email.
L-0 2	<code>InscriptionService</code> n'est pas transactionnel	Si la création du profil échoue après celle du compte, le compte reste en base.
L-0 3	<code>nombre_max_mentores</code> est informatif	Le mentor déclare sa capacité, mais le système ne bloque pas automatiquement une nouvelle demande au-delà de ce plafond.
L-0 4	Coverage de tests automatisés limitée	6 tests unitaires couvrent <code>PasswordUtil</code> (hachage et vérification). Les DAO, services et contrôleurs ont été validés manuellement.
L-0 5	Pas d'envoi d'emails	Toutes les notifications sont internes à l'application (table <code>notification</code>).

N°	Limite	Explication
L-0 6	Pas de visioconférence	La modalité <code>EN_LIGNE</code> repose sur un lien fourni par le mentor.
L-0 7	Absence de cache applicatif	Chaque écran interroge la base ; suffisant à l'échelle du projet.

14. Planning réalisé

Semaine	Travail prévu	Livraison
1	MEA, MLD, base MySQL, projet Java/JavaFX, test JDBC	Schéma et connexion opérationnelle
2	Authentification, inscription, utilisateurs, UFR, filières, niveaux, matières	Inscription et référentiel gérables
3	Recherche de mentor, demandes, validation/refus, séances	Workflow de mentorat fonctionnel
4	Évaluations, corrections, tests, préparation de la démonstration	Version 1.0

15. Livrables

Livable	Emplacement
Code source complet	<code>src/main/java</code> , <code>src/main/resources</code>
Scripts de base de données	<code>database/schema.sql</code> , <code>database/data.sql</code> , <code>database/migration_00*.sql</code>
Documentation	<code>README.md</code> , <code>CAHIER_DE_CHARGE.md</code>
Dépôt versionné	GitHub — branche <code>main</code>
Fichier d'installation	Non produit (application lancée par Maven)

16. Critères d'acceptation

Une fonctionnalité est considérée comme livrée quand :

1. le projet compile sans erreur ;
2. la fonctionnalité est opérationnelle depuis l'écran concerné ;
3. elle communique correctement avec MySQL ;
4. les cas d'erreur principaux sont gérés par un message lisible ;
5. elle a été testée manuellement ;
6. le code est poussé sur GitHub ;
7. la Pull Request a été relue par un autre membre.

État de recette de la version 1.0

Contrôle	Résultat
Compilation	sans erreur
Tests unitaires	6/6 réussis
Chargement des 15 écrans FXML	15/15, aucune exception
Boutons au texte tronqué	0
Liens de navigation testés	conformes
Boutons testés au picking réel	conformes
Erreurs de compilation	0

17. Organisation de l'équipe

Membre	Responsabilités
Membre 1 — Chef de projet / Architecture, GitHub, structure, <code>Main.java</code> , navigation, intégration, Intégrateur	revue de code, tests globaux
Membre 2 — Base de données / JDBC	MEA, MLD, <code>schema.sql</code> , <code>data.sql</code> , <code>DatabaseConnection</code> , requêtes
Membre 3 — Interface JavaFX	FXML, CSS, écrans, tableaux de bord, formulaires, ergonomie
Membre 4 — Logique métier / Contrôleurs	services, authentification, rôles, demandes, séances, évaluations

Processus Git : un dépôt unique, `main` ne contient que du code stable. Chaque membre travaille sur une branche dédiée à sa fonctionnalité, puis soumet une Pull Request relue avant fusion dans `main`.

Projet réalisé dans le cadre du cours de Java avancé — Université Alioune Diop de Bambey.